

TD de Programmation Orientée Objet

Licence d'Informatique*

Semaine du 23 février 2026

Design pattern Decorator

Dans le TD précédent nous avons construit la classe `FilterArrayIterator` qui nous permet de filtrer un itérateur sur un tableau en fonction d'un prédicat (instance d'une classe implémentant l'interface `Predicate<E>`). En utilisant le design pattern Decorator nous allons voir comment définir une classe `FilterIterator` qui permettrait de transformer (ou décorer) n'importe quel itérateur en un itérateur filtrant ses éléments.

Exercice 1 Définition de la classe `FilterIterator<E>`

La classe `FilterIterator<E>` possède un unique constructeur prenant en paramètre un itérateur quelconque ainsi qu'une instance d'une classe implémentant l'interface `Predicate<E>`.

1. En vous inspirant du code de la classe `FilterArrayIterator` définissez dans la classe `FilterIterator<E>` une méthode `void findNext()` qui affecte à l'attribut `nextElt`, le premier élément – après la position courante de l'itérateur – qui satisfait le prédicat et la valeur `true` à l'attribut `nextFound`. Dans le cas où un tel élément n'existe pas `findNext()` affecte la valeur `false` à l'attribut `nextFound`.

```
public class FilterIterator<E> implements Iterator<E> {
    private Iterator<? extends E> supportIter;
    private Predicate<? super E> predicate;
    private boolean nextFound;
    private E nextElt;
    public FilterIterator(Iterator<? extends E> iter,
                          Predicate<? super E> p) {
        // ...
    }
    private void findNext() {
        // A completer
    }
    // Autres methodes
}
```

2. Complétez le code de la classe en définissant le constructeur et les méthodes `boolean hasNext()` et `E next()`.
3. Testez la classe à l'aide de la méthode `main` suivante :

```
public static void main(String[] args) {
    List<String> lStr = Arrays.asList(null, "aa", null, "bb",
                                      "cc", null);

    Iterator<String> iter
        = new FilterIterator<>(lStr.iterator(), s -> s != null);
    while (iter.hasNext()) {
        System.out.print(": " + iter.next());
    }
    System.out.println();
    // Affichage: :aa:bb:cc
}
```

*Marc Champesme – Département d'Informatique – Institut Galilée

```
}
```

4. Complétez le code suivant afin de pouvoir utiliser un itérateur filtrant les `ObjetZork` de poids supérieur à 10, comme dans l'exemple suivant :

```
ObjetZork oz1 = new ObjetZork("oz1", 5, true),
oz2 = new ObjetZork("oz2", 45, true),
oz3 = new ObjetZork("oz3", 7, false),
oz4 = new ObjetZork("oz4", 15, false),
oz5 = new ObjetZork("oz5", 11, true),
ObjetZork[] tabOz = {oz1, oz2, oz3, oz4, oz5};
// a completer
Iterator<ObjetZork> iter = new FilterIterator<>(?);
while (iter.hasNext()) {
    System.out.print(": " + iter.next().getNom());
}
System.out.println();
// Affichage: :oz2:oz4:oz5
```

Exercice 2 Définition d'un nouveau décorateur.

La bibliothèque de classe Java définit la *functional interface* `Function` de la manière suivante :

```
public interface Function<T, R> {
    /**
     * Applies this function to the given argument.
     * @param t the function argument
     * @return the function result
     */
    R apply(T t);
}
```

1. Définissez une nouvelle classe `NameFunction` héritant de `Function<ObjetZork, String>` et telle que la méthode `apply` renvoie le nom d'un `ObjetZork`. Utilisez le pattern Singleton pour garantir qu'une seule instance de la classe sera créée. Vous pourrez tester votre code avec le code suivant.

```
ObjetZork oz1 = new ObjetZork("oz1name", 5, true),
oz2 = new ObjetZork("oz2name", 45, true),
oz3 = new ObjetZork("oz3name", 7, false),
oz4 = new ObjetZork("oz4name", 15, false),
oz5 = new ObjetZork("oz5name", 11, true),
ObjetZork[] tabOz = {oz1, oz2, oz3, oz4, oz5};
Iterator<ObjetZork> iter = new ArrayIterator<>(tabOz);
NameFunction f = ???
while (iter.hasNext()) {
    System.out.print(": " + f.apply(iter.next()));
}
System.out.println();
// Affichage: :oz1name:oz2name:oz3name:oz4name:oz5name
```

2. En utilisant la *functional interface* `Function` définissez une nouvelle classe `MappingIterator<T, E>` qui permettrait de *décorer* un itérateur d'éléments de type `T` en remplaçant chacun de ses éléments par le résultat (de type `E`) de l'application de la fonction à cet élément, comme dans le code suivant :

```
ObjetZork oz1 = new ObjetZork("oz1name", 5, true),
oz2 = new ObjetZork("oz2name", 45, true),
oz3 = new ObjetZork("oz3name", 7, false),
oz4 = new ObjetZork("oz4name", 15, false),
oz5 = new ObjetZork("oz5name", 11, true);
ObjetZork[] tabOz = {oz1, oz2, oz3, oz4, oz5};
```

```

NameFunction f = NameFunction.theInstance;
Iterator<String> iter
    = new MappingIterator<>(new ArrayIterator<>(tabOz), f);
while (iter.hasNext()) {
    System.out.print(": " + iter.next());
}
System.out.println();
// Affichage: :oz1name:oz2name:oz3name:oz4name:oz5name

```

Exercice 3 Usage combiné de plusieurs décorateurs

En utilisant les classes `ArrayIterator`, `FilterIterator` et `MappingIterator`, construisez un itérateur permettant, à partir d'un tableau d'`ObjetZork`, d'afficher les noms des `ObjetZork` qui commencent par le préfixe "Pi", en complétant le code ci-après. Vous pourrez utiliser la méthode suivante de la classe `String` :

boolean startsWith(String prefix) Tests if this string starts with the specified prefix.

Complétez le code suivant :

```

ObjetZork oz1 = new ObjetZork("Epee", 50, true),
oz2 = new ObjetZork("Pizza", 5, true),
oz3 = new ObjetZork("Tresor", 70, true),
oz4 = new ObjetZork("Parchemin", 2, false),
oz5 = new ObjetZork("Tableau", 110, true),
oz6 = new ObjetZork("Pistolet", 11, true);
ObjetZork[] tabOz = {oz1, oz2, oz3, oz4, oz5};
// A completer
Iterator<String> iterStr = ???
while (iter.hasNext()) {
    System.out.print(": " + iter.next());
}
System.out.println();
// Affichage: :Pizza:Pistolet

```

Exercice 4 Construction d'un itérateur fabriquant des `ObjetZork`.

En utilisant les classes `FilterIterator` et `MappingIterator`, on souhaite construire un itérateur dont les éléments sont des `ObjetZork` créés à partir d'une liste de noms (i.e. instances de la classe `String`), pouvant contenir la valeur `null`. Les valeurs des attributs poids et `estTransportable` seront générés aléatoirement en utilisant les méthodes suivantes de la classe `Random` (package `java.util`) :

int nextInt(int bound) Returns a pseudorandom, uniformly distributed int value between 0 (inclusive) and the specified value (exclusive).

boolean nextBoolean() Returns the next pseudorandom, uniformly distributed boolean value from this random number generator's sequence.

Complétez le code suivant :

```

public static void main(String[] args) {
    List<String> lStr = Arrays.asList(null, "Pizza", "Tresor",
                                     "Table", "Chaise", null, "Livre",
                                     "Coffre", "Clef", "Gateau");

    Random randGen = new Random();
    Iterator<String> iterStr = lStr.iterator();
    // A completer
    Iterator<ObjetZork> iterOz = ???
    List<ObjetZork> lOz = new ArrayList<ObjetZork>();
    while (iterOz.hasNext()) {
        lOz.add(iterOz.next());
    }
    System.out.println(lOz);
}

```